SYSTEM DESIGN SERIES • SDE1 GUIDE

URL Shortener

System Design

Study Material & Interview Preparation Guide

Base62 Encoding | Hashing | Redis Caching | ZooKeeper | Snowflake IDs

7	50+	15+	Real
Chapters	Q&A	Diagrams	Examples

TABLE OF CONTENTS

01	What is a URL & Why Shorten It?	Fundamentals
02	Problems with Long URLs	Problem Statement
03	How URL Shorteners Work	Core Mechanics
04	Why Not Hash the String?	Hashing vs Encoding
05	Base62 Encoding — Deep Dive	Encoding Algorithm
06	Unique ID Generation Methods	Distributed Systems
07	ZooKeeper & Distributed Coordination	Infrastructure
08	Issues & Edge Cases	Engineering Challenges
09	System Design — Full Architecture	High-Level Design
10	Interview Questions & Model Answers	Interview Prep

01 WHAT IS A URL & WHY SHORTEN IT?

A **URL (Uniform Resource Locator)** is the web address you type in a browser to access a resource on the internet. Every URL has a structure:

```
https://www.amazon.in/dp/B09G3HRMVB?ref=ppx_yo2ov_dt_b_fed_asin_title
        &pd_rd_w=abc123&pf_rd_p=xyz789&linkCode=sl1&tag=mystore-21

 Protocol   Domain          Path        Query Parameters
 [https]  [amazon.in]  [/dp/B09G3]  [?ref=...&pd_rd=...&tag=...]
```

The URL above is **137 characters long**. This is common for e-commerce, analytics, and content platforms that embed tracking parameters in URLs.

What a URL Shortener Does:

```
LONG  →  https://www.amazon.in/dp/B09G3HRMVB?ref=ppx_yo2ov_dt_b...
                              ↓
SHORT →  https://amzn.to/3xKpQ2

When user visits short URL → server looks up mapping → HTTP redirect
```

Real-World Service	Example Short URL	Use Case
Bit.ly	bit.ly/3xKpQ2	General purpose shortener
TinyURL	tinyurl.com/y4x	Simple shortening
amzn.to	amzn.to/3AbcDef	Amazon product links
t.co	t.co/AbCdEf	Twitter auto-shortener
youtu.be	youtu.be/dQw4w	YouTube video links
rb.gy	rb.gy/xyz123	Campaign tracking

02 PROBLEMS WITH LONG URLS

Long URLs create multiple real-world problems across different platforms and use cases:

Not Shareable

Hard to copy-paste, especially on mobile keyboards
Easy to mistype even one character, breaking the link
Impossible to share verbally or write on a whiteboard

Platform Char Limits

SMS: 160 character limit — a URL can consume it entirely
Twitter/X: 280 char limit — long URLs eat into tweet space
Email subject lines get truncated with long URLs

Looks Untrustworthy

URLs with ?ref=...&utm_source=...&gclid=... look like spam
Users hesitate to click unfamiliar long query strings
Phishing attacks often hide behind long obfuscated URLs

QR Codes Degrade

Longer URL = more data = denser QR code
Dense QR codes are harder to scan, especially when printed small
A 6-char short URL produces a clean, scannable QR code

No Analytics

You cannot track who clicked, when, from where, on what device
Marketers need per-campaign click data
Short URLs act as tracking middleware before redirect

03 HOW URL SHORTENERS WORK

Step 1 — User submits a long URL

A POST request is made to the API with the long URL as the body.

Step 2 — System generates a unique short code

A unique integer ID is generated and encoded to Base62 (e.g., 12345 → '3d7').

Step 3 — Mapping is stored

```
Table: url_mapping
┌─────────────┬──────────────────────┬────────────┬────────────┐
│ short_code  │ long_url             │ created_at │ expires_at │
├─────────────┼──────────────────────┼────────────┼────────────┤
│ 3xKpQ2      │ https://amazon.in/dp/... │ 2024-01-01 │ NULL       │
│ aB9mTz      │ https://github.com/...   │ 2024-01-02 │ 2025-01-02 │
└─────────────┴──────────────────────┴────────────┴────────────┘
```

Step 4 — User receives the short URL

The server returns the short URL: **https://bit.ly/3xKpQ2**

Step 5 — Someone clicks the short URL (Redirect Flow)

```
User clicks bit.ly/3xKpQ2
        ↓
Server extracts code '3xKpQ2'
        ↓
Check Redis cache (microseconds)
   → HIT  → return long URL immediately
   → MISS → query database → store in cache → return long URL
        ↓
HTTP 302 Redirect → Browser navigates to long URL
```

Redirect Type	HTTP Code	Browser Caches?	Analytics Tracked?	Use When
Permanent	301	Yes	No (after cache)	Static pages, SEO
Temporary	302	No	Yes (always)	URL shorteners, tracking

Key Insight

URL shorteners use HTTP 302 (temporary redirect) so every click passes through the server — enabling click counting, geo-tracking, device analytics, and A/B testing per campaign link.

04 WHY NOT HASH THE STRING?

Hashing (MD5, SHA-256) seems natural: just hash the long URL and use the first 6 characters. Let's examine why this approach fails:

Problem 1: Hash Collisions

```
URL A → MD5 → 'a3f9bc21d8...' → first 6 chars → 'a3f9bc'
URL C → MD5 → 'a3f9bc99ef...' → first 6 chars → 'a3f9bc'  ← COLLISION!

Two different URLs now map to the SAME short code → System broken!
```

Problem 2: Collision Resolution is Expensive

To fix collisions, you must check the DB for each generated code: 'Is this taken?' If yes, modify input and re-hash. This creates a slow retry loop that worsens under load.

Problem 3: Non-Sequential, Hard to Manage

Hash outputs have no ordering. You can't say 'give me all URLs shortened today' from a hash alone. It gives zero operational visibility.

Problem 4: Predictability Paradox

If two users shorten the same URL, they get the same hash (a conflict if per-user tracking is needed). But if salt is added, the same URL gets different hashes — defeating deduplication.

Approach	Collision Risk	Manageable	Fast at Scale	Analytics Friendly
MD5/SHA (truncated)	HIGH	No	No (retry loop)	No
Base62 (unique ID)	ZERO	Yes	Yes	Yes
UUID	Very Low	No (too long)	Yes	No

Bottom Line

Hashing is a one-way function designed for integrity checking, not unique key generation. Base62 encoding of a unique ID avoids ALL collision problems by construction — no two IDs are the same.

05 BASE62 ENCODING — DEEP DIVE

Base62 is a numeral system using **62 characters**: digits 0–9, lowercase a–z, and uppercase A–Z. It converts a large integer ID into a compact, URL-safe string.

```
Alphabet (index 0 to 61):
'0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ'

Index:   0  1  2  ...  9  10  11  ... 35  36  37  ... 61
Char:    0  1  2  ...  9   a   b  ...  z   A   B  ...  Z
```

Encoding Algorithm (ID → Short Code)

Repeatedly divide the ID by 62. Each remainder is an index into the alphabet. Read remainders from bottom to top.

```
Example: Encode ID = 125000

125000 ÷ 62 = 2016  remainder 8   → alphabet[8]  = '8'
  2016 ÷ 62 =   32  remainder 32  → alphabet[32] = 'w'
    32 ÷ 62 =    0  remainder 32  → alphabet[32] = 'w'

Read bottom to top → Short code = 'ww8'
```

Decoding Algorithm (Short Code → ID)

```
Decode 'ww8'  →  index('w')=32, index('w')=32, index('8')=8

= 32 × 62² + 32 × 62¹ + 8 × 62■
= 32 × 3844 + 32 × 62 + 8
= 123008 + 1984 + 8
= 125000  ✓  (original ID recovered!)
```

Capacity by Code Length

Code Length	Combinations	Enough For
5 chars	62■ = 916 Million	Small apps
6 chars	62■ = 56 Billion	Medium platforms
7 chars	62■ = 3.5 Trillion	Large scale (recommended)
8 chars	62■ = 218 Trillion	Enterprise / Future-proof

Why Base62 over Base64?

Base64 uses + and / which are special characters in URLs and require percent-encoding (%2B, %2F). Base62 uses only alphanumeric characters that are safe in URLs without any encoding — simpler and cleaner.

06 UNIQUE ID GENERATION METHODS

The core challenge: generate IDs that are **unique across all servers simultaneously**, without coordination overhead. Here are the main approaches:

Method 1: Auto-Increment (Single DB)

```
DB: ID = 1, 2, 3, 4, 5...
Simple AUTO_INCREMENT primary key.
Each insert gets a new unique ID.
```

- Pros: Simple, guaranteed unique, ordered
- Cons: Single point of failure, write bottleneck,
- IDs are sequential (enumerable by attackers)

Method 2: UUID (Random)

```
550e8400-e29b-41d4-a716-446655440000
128-bit random identifier.
Generated independently on each server.
```

- Pros: No central coordinator, very low collision prob
- Cons: Too long for short codes (must truncate = risk),
- Not sortable, not human-friendly

Method 3: Snowflake ID (Twitter)

```
[ 41-bit timestamp | 10-bit machine ID | 12-bit seq ]
64-bit integer. Globally unique, time-sortable.
Each server uses its own machine ID independently.
```

- Pros: Distributed, fast (4096 IDs/ms per server),
- Sortable, no DB needed per generation
- Cons: Requires clock sync (NTP), clock skew is risky

Method 4: Pre-Generated Pool

```
Generate batch of IDs in advance, store in Redis set.
Each server pops an ID when needed.
Background job refills the pool.
```

- Pros: No real-time coordination, extremely fast reads
- Cons: Complex to maintain, IDs wasted on server crash,
- Pool refill logic adds operational overhead

Method 5: Token Range (via ZooKeeper)

```
Server 1: IDs 1–1,000,000
Server 2: IDs 1,000,001–2,000,000
ZooKeeper assigns non-overlapping ranges.
```

- Pros: No per-ID coordination, fast within range,
- Clean failure recovery (unused range is lost only)
- Cons: Need ZooKeeper coordinator, wasted IDs on crash

Comparison Summary

Method	Unique?	Distributed?	Sequential?	Complexity
Auto-Increment	Yes	No	Yes (risky)	Low
UUID	Yes (prob)	Yes	No	Low
Snowflake ID	Yes	Yes	Yes (safe)	Medium
Pre-Gen Pool	Yes	Yes	No	High
Token Range	Yes	Yes	Yes	Medium

07 ZOOKEEPER & DISTRIBUTED COORDINATION

Apache ZooKeeper is a **distributed coordination service** — a highly reliable shared notepad that all servers in a cluster can read and write to atomically.

ZAB Protocol (ZooKeeper Atomic Broadcast)

```
One node elected as LEADER
        ↓
All writes go to LEADER
        ↓
LEADER broadcasts to FOLLOWERS
        ↓
Majority (quorum) acknowledge → write committed
        ↓
If leader dies → ZAB auto-elects new leader
```

Znode Tree Structure

```
/                              (root)
├── /url-shortener
│   ├── /servers
│   │   ├── /server-1          → 'range: 1–1,000,000'
│   │   ├── /server-2          → 'range: 1,000,001–2,000,000'
│   │   └── /server-3          → 'range: 2,000,001–3,000,000'
│   └── /leader                → 'server-1'
```

Znode Type	Behavior	URL Shortener Use
Persistent	Stays until explicitly deleted	Store ID range assignments
Ephemeral	Auto-deleted on client disconnect	Leader election, server health
Sequential	ZK appends auto-incrementing number	Generate ordered range IDs

How ZooKeeper Solves ID Generation

```
1. Server boots → contacts ZooKeeper
2. ZK creates: /ranges/range-0000000003 → assigned to Server 3
3. Server uses IDs 2,000,001–3,000,000 locally (no coordination)
4. Range exhausted → Server requests new range from ZK
5. No two servers ever share a range → zero collision
```

ZooKeeper Quorum (Ensemble)

ZooKeeper runs as an odd-numbered cluster to always have a clear majority:

Ensemble Size	Failures Tolerated	Quorum Needed	Production?

3 nodes	1 failure	2/3 agree	Small teams
5 nodes	2 failures	3/5 agree	Recommended
7 nodes	3 failures	4/7 agree	High availability

ZooKeeper Guarantees

- **Sequential Consistency:** Updates appear in the order they were sent by clients
- **Atomicity:** A write either fully succeeds or fully fails — never partial
- **Single System Image:** Client sees same view regardless of which ZK server it connects to
- **Reliability:** Once committed, updates persist even if servers fail
- **Timeliness:** Clients see updates within a bounded (fast) time

08 ISSUES & EDGE CASES

Building a URL shortener at scale surfaces many non-obvious engineering challenges:

Race Condition on ID Generation

Two servers generate the same ID at the same millisecond
Solution: Snowflake machine_id ensures server-level uniqueness
Or: Use DB transactions / Redis atomic INCR

Custom Alias Conflicts

User wants bit.ly/my-brand — conflicts with auto-generated codes
Solution: Maintain a reserved/blacklist table in DB
Check alias availability before accepting user request

URL Enumeration Attack

Sequential IDs (1,2,3...) allow attackers to scrape all URLs
Solution: Shuffle ID space, apply Base62, add random salt
Or: Rate limit requests to the redirect endpoint

Redirect Chain (Short URL of Short URL)

User shortens a bit.ly URL → creates a chain of redirects
Solution: Detect if input is already a short URL, resolve first
Limit redirect chain depth to prevent infinite loops

URL Expiry & Cleanup

Expired URLs must be cleaned from DB and cache
Solution: Add expires_at column, use a background cron job
Also remove from Redis cache to prevent stale data

Cache Invalidation

Long URL changes → cache still serves old value
Solution: Use TTL-based cache expiry (e.g., 24h in Redis)
Or: Explicit cache bust when long URL is updated in DB

Malicious / Invalid URLs

Users submit phishing, malware, or NSFW URLs
Solution: Validate URL format with regex, check against
Google Safe Browsing API or internal blacklist on submission

Database Bottleneck on Reads

Billions of redirects/day hit the same DB → bottleneck

Solution: Cache hot short codes in Redis (LRU eviction)

Add DB read replicas for load distribution

09 FULL SYSTEM ARCHITECTURE

Here is the complete high-level design of a production-grade URL shortener:

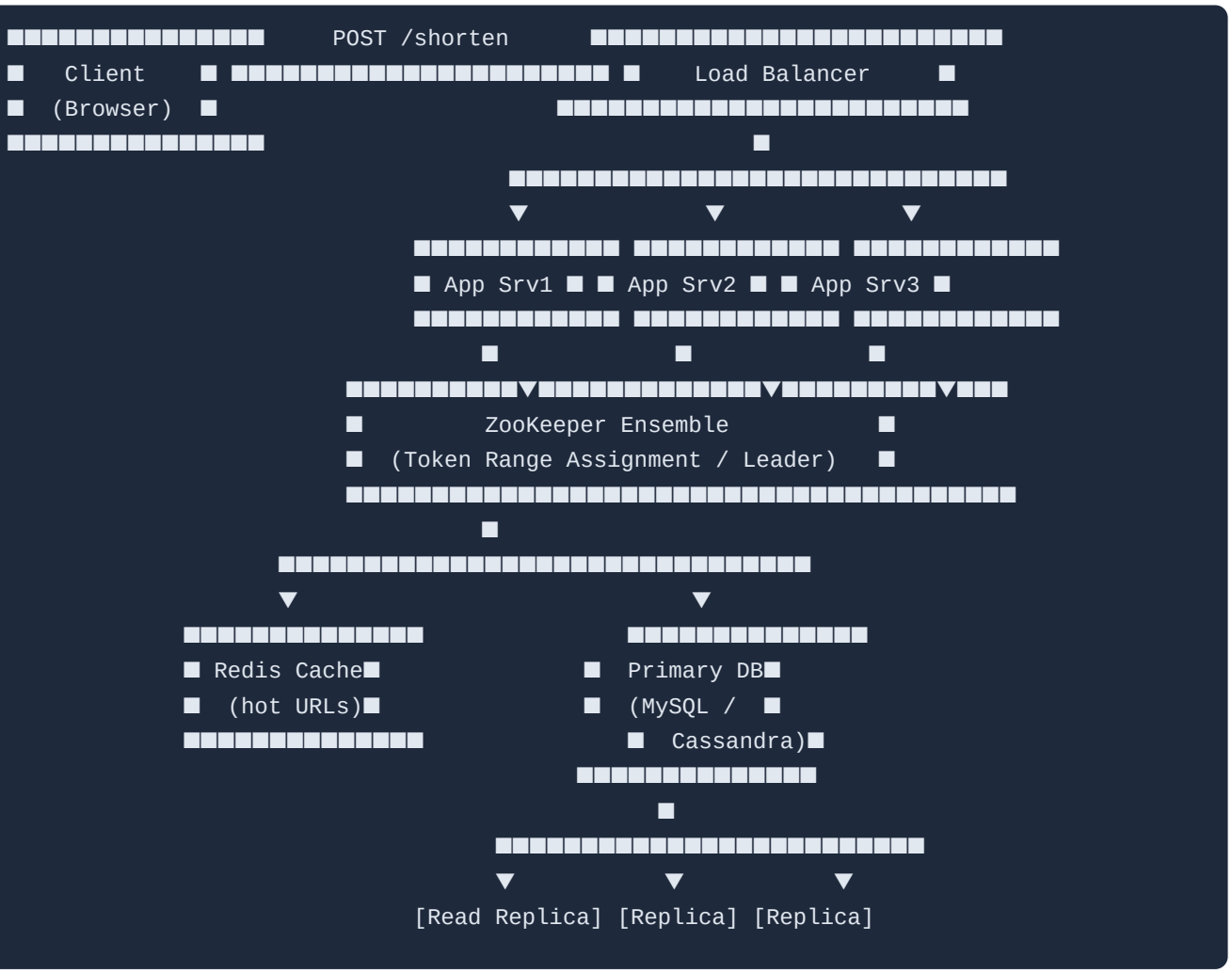
Back-of-Envelope Estimation

```
Assumptions:
  • 100 million URLs shortened per day
  • 10:1 read-to-write ratio → 1 billion redirects/day

Writes/sec:  100M ÷ 86,400  ≈  1,200 writes/sec
Reads/sec:   1B   ÷ 86,400  ≈  11,500 reads/sec

Storage/URL: ~500 bytes
Daily:       100M × 500B = 50 GB/day
Yearly:      50 GB × 365 ≈ 18 TB/year
```

Component Architecture



REST API Design

```
POST   /api/shorten
  Body:     { 'long_url': 'https://...', 'custom_alias': 'optional' }
  Response: { 'short_url': 'https://bit.ly/3xKpQ2', 'expires_at': '...' }
  Codes:    201 Created | 400 Bad Request | 409 Conflict (alias taken)

GET    /{short_code}
  Response: HTTP 302 Redirect to long_url
  Codes:    302 Found | 404 Not Found | 410 Gone (expired)

GET    /api/stats/{short_code}
  Response: { 'clicks': 14823, 'created_at': '...', 'top_country': 'IN' }

DELETE /api/{short_code}
  Response: 200 OK | 403 Forbidden | 404 Not Found
```

Database Schema

```
CREATE TABLE url_mapping (
    id           BIGINT      PRIMARY KEY AUTO_INCREMENT,
    short_code   VARCHAR(10) UNIQUE NOT NULL,
    long_url     TEXT        NOT NULL,
    user_id      BIGINT      REFERENCES users(id),
    created_at   DATETIME    DEFAULT NOW(),
    expires_at   DATETIME    NULL,
    click_count  INT         DEFAULT 0
);
CREATE INDEX idx_short_code ON url_mapping(short_code);
```

10 INTERVIEW QUESTIONS & MODEL ANSWERS

The following questions are commonly asked at SDE1 level interviews. Model answers follow each question.

Q1. What is a URL shortener and why do we need it?

A URL shortener converts a long URL into a compact short URL. We need it because long URLs are hard to share (especially on SMS and social media with character limits), look untrustworthy, break in emails, create dense QR codes, and make analytics difficult. Short URLs act as a redirect layer that also enables click tracking.

Q2. Why is hashing (MD5/SHA) a bad approach for generating short codes?

MD5/SHA produce fixed-length hashes, but when you truncate to 6-7 characters, different URLs can produce the same prefix — this is a collision. Resolving collisions requires DB lookups and retries, which is slow at scale. Additionally, hashes give no ordering or operational visibility, making system management harder. Base62 encoding of a unique ID has zero collisions by construction.

Q3. Explain Base62 encoding. Why use it over Base64?

Base62 uses 62 alphanumeric characters (0-9, a-z, A-Z) to encode a large integer into a short string. Divide the integer by 62 repeatedly; each remainder maps to a character. Read bottom-to-top for the code. Base64 adds '+' and '/' which are special URL characters requiring percent-encoding. Base62 is purely alphanumeric — safe in URLs without encoding.

Q4. What is the difference between HTTP 301 and HTTP 302 redirects?

301 is a permanent redirect — the browser caches it, so future visits skip the server entirely. This saves server load but means you lose click analytics after first visit. 302 is a temporary redirect — the browser always contacts the server, so you track every click. URL shorteners use 302 because analytics (click counts, geo, device) are a core feature.

Q5. How would you design the database schema for a URL shortener?

Main table url_mapping with columns: id (BIGINT, auto-increment PK), short_code (VARCHAR, unique index), long_url (TEXT), user_id (FK), created_at (DATETIME), expires_at (DATETIME, nullable), click_count (INT). Index on short_code for O(1) lookup. Separate analytics table for detailed click events if needed.

Q6. How does caching help in a URL shortener? What would you cache?

URL shorteners are read-heavy (10:1 ratio). For every redirect, the server does a DB lookup. Caching frequently-accessed short codes in Redis means the DB is only hit on cache misses. Cache the short_code → long_url mapping with an LRU eviction policy. Popular short URLs stay warm in cache, reducing DB load by 90%+.

Q7. What is a race condition in URL shortener and how do you prevent it?

A race condition occurs when two servers simultaneously generate the same ID and try to insert the same short_code into the DB. Prevention: Use DB unique constraints (INSERT will fail for duplicate), use atomic Redis INCR for counter generation, use Snowflake IDs (machine_id ensures server-level uniqueness), or use ZooKeeper for distributed token range assignment.

Q8. What is Snowflake ID and how does it solve distributed ID generation?

Snowflake is a 64-bit integer composed of: 41 bits for millisecond timestamp, 10 bits for machine/server ID, 12 bits for sequence number within a millisecond. The machine ID ensures two servers never produce the same ID even at the exact same millisecond. The sequence handles up to 4096 IDs per millisecond per server. Result: globally unique, time-sortable, no DB coordination needed.

Q9. What is ZooKeeper and how is it used in a URL shortener?

ZooKeeper is a distributed coordination service. In a URL shortener, it assigns non-overlapping ID ranges to servers (Token Range method). Server 1 gets 1-1M, Server 2 gets 1M-2M, etc. Each server uses its range independently — no coordination per ID. ZooKeeper also handles leader election and failure detection via ephemeral znodes. It runs as a 3 or 5-node ensemble for fault tolerance.

Q10. SQL or NoSQL for URL shortener — which would you choose and why?

For the core mapping (short_code → long_url), either works. SQL (MySQL/PostgreSQL) gives strong consistency and easy querying. NoSQL (Cassandra, DynamoDB) gives better horizontal scalability and is optimized for key-value lookups. At large scale, Cassandra is preferred because the primary access pattern is a single key lookup (short_code), which NoSQL handles extremely efficiently with no joins required.

Q11. How would you handle URL expiry?

Add an expires_at column to the URL mapping table. On every redirect request, check if expires_at is in the past — if so, return 410 Gone. Run a background cron job (every hour/day) to delete expired rows from DB and invalidate their Redis cache entries. Expired short codes can be reused after deletion, freeing up the namespace.

Q12. How do you prevent enumeration attacks on short URLs?

Sequential IDs (1,2,3...) allow attackers to scrape all shortened URLs by iterating. Solutions: (1) Base62 encode large IDs so the output looks non-sequential even if the IDs are sequential. (2) Add a random component or shuffle the ID space. (3) Rate-limit the redirect endpoint per IP. (4) Require authentication for sensitive links. (5) Use URL expiry so old links don't stay forever.

Q13. Walk me through the full flow when a user clicks a short URL.

1) Browser sends GET bit.ly/3xKpQ2. 2) DNS resolves bit.ly to load balancer IP. 3) Load balancer routes to an app server. 4) App server extracts code '3xKpQ2'. 5) Checks Redis cache — if HIT, gets long URL immediately. 6) If MISS, queries DB for short_code = '3xKpQ2'. 7) Stores result in Redis cache. 8) Asynchronously logs click event to Kafka for analytics. 9) Returns HTTP 302 with Location: long_url. 10) Browser follows redirect to long URL.

Q14. How would you implement the Base62 encode/decode in code?

Encoding: Set alphabet = '0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ'. While ID > 0: append alphabet[ID % 62] to result, set ID = ID // 62. Reverse the result string. Decoding: For each character in code, find its index in alphabet. Result = sum(index * 62^position) for each character from right to left.

Q15. What would you do if two users want the same custom alias?

Return a 409 Conflict HTTP response to the second user. Before accepting a custom alias, check the url_mapping table: SELECT COUNT(*) WHERE short_code = alias. If count > 0, reject with error message 'Alias already taken, please choose another.' Also maintain a blacklist of reserved words (api, admin, health, stats, etc.) that can never be used as custom aliases.

Quick Reference Cheat Sheet

Concept	Key Point	Remember
Base62	62 chars: 0-9, a-z, A-Z	7 chars = 3.5 trillion URLs
Why not hash?	Truncated hashes collide	Base62(unique ID) = zero collision
301 vs 302	301 cached (no analytics), 302 not (analytics)	Use 302 for URL shorteners
Snowflake ID	41-bit time + 10-bit machine + 12-bit seq	4096 IDs/ms per server
ZooKeeper	Distributed coordinator, ZAB consensus	Assigns non-overlapping ID ranges
Redis Cache	LRU eviction, in-memory, microsecond reads	Cache short_code → long_url
Read:Write ratio	~10:1 (reads vastly dominate)	Design for read-heavy system
Race Condition	Two servers same ID simultaneously	Unique constraint + Snowflake
URL Expiry	expires_at column + background cron job	Return 410 Gone for expired URLs